

CSDC102 - FINAL PROJECT - ROAD CROSSING CHALLENGE

DOCUMENTATION OF LINKED LIST & FILE I/O USAGE

Members: Angelo D. Oduca, August John R. Reganit, John David B. Bobos, Tristan Dwayne B. Opeña, Lester Aeron V. Nosil

Created Struct Node, to manage Obstacles (i.e., trucks & logs)

```
struct Node{
    Obstacle obstacle;
    Node *next;
};
```

Created void addObstacle() - making every spawned truck or log becomes a new linkedlist node

```
Node *newNode = new Node;
```

```
temp->next = newNode;
```

Used Linked List in the functions below, in order to manage when a Player is passing obstacles, Hitting Obstacles, or Using Logs.

displayMap(), moveObstacles(), checkCollision(), moveWithLog(), isPlayerOnLog(), landOccupied()

In **displayMap()**, Line 403 - 407, it moves the node to the next node in the linked list using temp = temp-> next.

In **moveObstacles()**, Line 345 - 386, it moves obstacles by updating their positions and decides if an obstacle should be removed.

In **checkCollision()**, Line 546 - 599, it checks the obstacle nodes to check whether the player collided with a truck '#' or is standing on a log '='.

In **moveWithLog()**, Line 482 - 512, it checks if the player is on a log '=', if it's true the player will move with the log.

In **isPlayerOnLog()**, Line 515 - 542, it continuously check if the players position matches any log '=' positions

In **landOccupied()**, Line 328 - 343, it checks current obstacle Nodes before spawning new ones. To prevent overlapping or connected obstacles

It is also used to remove Nodes in the linked list, in moveObstacles(),

```
delete toDelete;
```

clearObstacles() also uses a linked list to free up memory when the gameloop ends or when the program ends.

```
while (head != nullptr)
```

The program also used fstream in the leaderboard() it uses the ifstream & ofstream.

```
ifstream file("leaderboard.txt");
```

First checking if the file exists, then proceeds to check if it is empty or not by using .peek().

If it isn't empty it proceeds to copy its content to variables player[].name and player[].score.

```
while (file >> players[count].name >> players[count].score)
```

When the player.life reaches 0, their score get to be inserted to the leaderboard.txt using ios::app which inserts the player.name and player.score to the bottom.

```
ofstream file("leaderboard.txt", ios::app);
```